

Integration Guide

Intelli-RFID Reader

Proposal Reference: NTPL/2026/DMRC/08/20-1 Date: 20-Aug-2026

NOUVORAIL TECH PRIVATE LIMITED
CIN U30202UP2023PTC187634

info@intellirail.in
www.intellirail.in

#615, Tower A, Noida One, Sector 62, Noida 201309

Table of Contents

1	About IntelliRail	4
1.1	Technology Competencies	4
2	Intelli-RFID Reader Overview	5
2.1	RFID Tunnel Manager	5
2.1.1	Super Fast Mode	5
2.1.2	Managed Reader Mode	5
2.1.3	Managed Writer Mode	5
3	REST API Definitions	7
3.1	Overview	7
3.1.1	Base URL	7
3.1.2	Conventions	7
3.1.3	The three modes	7
3.1.4	Why the expected quantity matters	7
3.1.5	EPC and TID	8
3.1.6	Errors	8
3.2	Authentication	8
3.2.1	Transport	9
3.2.2	The API key	9
3.2.3	Scopes	9
3.2.4	Authentication failures	9
3.2.5	Authenticating the reader to the WMS	10
3.2.6	Getting and rotating a key	10
3.3	Super Fast Mode	10
3.3.1	Arming the reader	11
3.3.2	What happens for each carton	11
3.3.3	The result callback	11
3.3.4	What your endpoint must do	13
3.3.5	Disarming and changing SKU	13
3.3.6	Errors specific to this mode	13
3.4	Managed Reading Mode	13
3.4.1	The timed flag	14

3.4.2	Synchronous form.....	14
3.4.3	Asynchronous form.....	14
3.4.4	Worked example — a short carton.....	15
3.4.5	Errors specific to this mode	15
3.5	Managed Writing Mode	15
3.5.1	Response fields	16
3.5.2	How the EPC is built	17
3.5.3	Serial allocation	17
3.5.4	Locking.....	17
3.5.5	Errors specific to this mode	17

1 About IntelliRail

IntelliRail provides the Next Generation Monitoring of Railway Assets like Rolling Stock, Track and Over Head Equipment. We design, develop, manufacture, and maintain Integrated Diagnostic Products that increase asset availability and improves the efficiency of the asset maintenance process.

Our solutions are backed by rigorous innovation and cutting-edge technology. However, we understand that providing the most advanced technology is not good enough. Therefore, our solutions have been designed with in-depth knowledge of the unique needs of our clients. And they are aligned with the environment and ecosystem in which they operate.

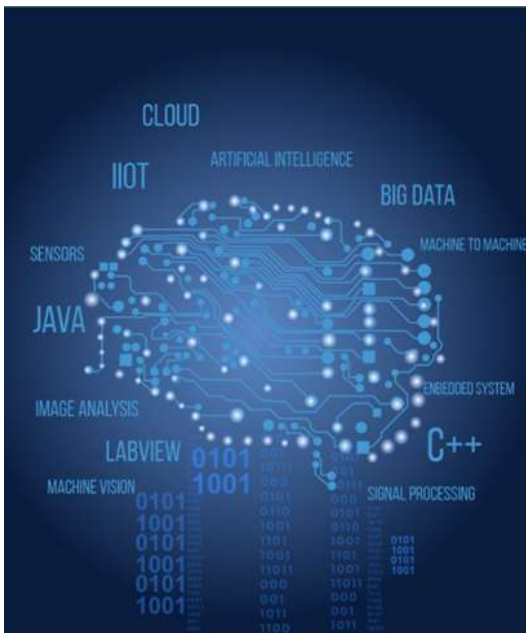
These Products and Systems include Wayside/Trackside Systems like Wheel Impact Load Detectors, Hot Box Detectors, ABD, HPDS, PVIS and On-Board Systems like On-Board Diagnostics for Locomotives, Generators, and various assets like TIMS, OHE Monitoring. These products and solutions help organizations make better engineering decisions and help improve their efficiencies with data and expert systems.

IntelliRail is founded and managed by an entrepreneurial team with several years of collective experience with leading multinationals across US, Europe and India. With a clear focus on quality and customer satisfaction backed by the skills of its people, IntelliRail consistently delivers the best-of-breed products, solutions and services.

IntelliRail is a brand owned and operated by NouvoRail Tech Pvt Ltd

1.1 Technology Competencies

We are an Original Equipment Manufacturer of Diagnostic Products for the Railway Industry. Our Areas of Expertise include:



- Machine Vision, AI and ML
- IIoT – low footprint systems integrated with Cloud
- Big Data and Smart Analytics
- Machine to Machine Communication
- Manufacturing, Installation and Deployment of On Board and Wayside systems
- Data Pipe Lining from Remote Locations to Cloud

2 Intelli-RFID Reader Overview

Intelli-RFID reader is a custom-built reader used for applications varying from train tracking in the railway organizations to managing tunnels in retail warehouses. Accessory boards are available to add on functionality from tracking railway wheels to controlling a conveyor belt.



2.1 RFID Tunnel Manager

The reader is completely managed by the customer's Warehouse Management System (WMS). The WMS makes REST API calls and receives calls from the Intelli-RFID reader to operate the tunnel.

The retail warehouse RFID tunnel has packages passing through a conveyor through 3 Zones:

- Entry Zone
- RFID Reading Zone
- Exit Zone

The WMS through a PLC, always monitors the movement of the package through the Entry Zone and the Exit Zone. The WMS can hand over the control of the movement of the package in the RFID Measurement Zone to the Intelli-RFID reader.

The WMS can operate the tunnel in three modes:

2.1.1 Super Fast Mode

The WMS enables this mode when a single SKU is to be passing through the tunnel for a certain period having the same number of products per box. To enable this mode, the REST API gives mode equal to superfast, the EAN number of the SKU being processed and the number of products per box in a single call, and then the Intelli-RFID reader takes over. In this mode, the RFID reader gets an input from an optical sensor when a carton arrives in the RFID reading zone. Within three to five seconds, Intelli-RFID Reader moves the palette out of the RFID reading zone and sends the TID and the EPC of each product within the package to the WMS via a REST API Post.

2.1.2 Managed Reader Mode

In this mode, the WMS manages the complete movement of the package and sends a REST API call to start the reading operation. This call includes the EAN and the quantity expected to be read from the carton. The API also passes a flag called timed equal to true. In which case, after the time provided is completed, the API returns the TID and EPC of all tags read, along with a flag stating whether the target quantity was met or not. If the same API call is made with timed equal to false then the RFID reader makes its best effort to read the tags within 3-5 seconds and makes a POST call with the TID and EPC of the tags read, along with the same flag stating whether the target quantity was met or not. In timed equal to false mode the server rest call URI also needs to be included for the POST call to be made

2.1.3 Managed Writer Mode

In this mode, the WMS makes a REST API call, passing the EAN and the quantity of tags to be written. The response returns the TID and EPC written to the tags found in the carton. The WMS can also pass

a quantity of tags to be written, in which case the response also returns whether the quantity was met or not as true or false.

3 REST API Definitions

3.1 Overview

This section defines the subset of the Intelli-RFID REST API that a Warehouse Management System needs in order to operate the tunnel. It covers the three operating modes described in Section 2.1, the authentication they require, and the request and response bodies for each. It assumes you can make HTTP requests and parse JSON. It does not assume any prior knowledge of RFID or of the reader hardware.

3.1.1 Base URL

Each reader serves one tunnel and exposes its own API at `http://<reader-host>:8081`. Substitute the address and port supplied by the site operator. The API is served over plain HTTP on the warehouse LAN; Section 3.2 sets out what that assumes about the network.

3.1.2 Conventions

Item	Convention
Encoding	JSON, UTF-8 throughout. Send Content-Type: application/json on any request that has a body.
Timestamps	ISO-8601 in UTC, for example 2026-08-27T09:14:02.118Z.
Durations	Milliseconds. Field names end in Ms.
Null fields	Omitted from responses rather than sent as null. Absent means not applicable, not zero.
Identifiers	EPC and TID are returned as uppercase hexadecimal strings with no separators.
Authentication	Every request carries an X-API-Key header. See Section 3.2.

3.1.3 The three modes

The modes differ in who moves the carton and how the result reaches the WMS. They are mutually exclusive: a reader is in one mode at a time.

Mode	Who moves the carton	How the result arrives	Use when
Super Fast	The reader, triggered by the optical sensor in the reading zone	The reader POSTs one result per carton to the WMS	A single SKU with a fixed count per box is running for a period
Managed Reading	The WMS, through its own PLC	Returned in the response, or POSTed to the WMS, depending on the timed flag	The WMS controls each carton and wants per-carton parameters
Managed Writing	The WMS	Returned in the response	Commissioning new tags onto stock

3.1.4 Why the expected quantity matters

Reading tags is not the difficult part. Knowing when every tag has been read is. Tags do not all answer at once: a passive tag replies only when it receives enough radio energy, and inside a densely packed carton the articles at the centre are shadowed by the ones around them. As the carton moves, the shadowing changes, and a tag that was silent for the first second becomes readable in the third.

A carton that reports 38 of its 40 articles is worse than useless, because nothing in the response distinguishes it from a correct read unless the API is designed to make that distinction explicit. Two articles have silently left the stock record, and nobody finds out until a cycle count months later.

Every mode therefore accepts the quantity expected per carton and returns an explicit `complete` flag. Supplying the expected quantity — from an ASN, a pick list or a packing manifest — is the single highest-value thing this integration can do, because it turns a tag count into a verifiable inventory and flags a short carton at the tunnel, while the goods are still physically present.

3.1.5 EPC and TID

Each article carries a passive UHF tag holding two identifiers, and all three modes return both.

Identifier	Meaning
EPC	Electronic Product Code. Written by the warehouse during commissioning and encoded as SGTIN-96, so it carries the article's EAN and a unique serial. This is the key your system joins on.
TID	Tag Identifier. Programmed by the tag manufacturer, globally unique and not rewritable. Useful for detecting a cloned or re-encoded tag, because two tags can carry the same EPC but never the same TID.

3.1.6 Errors

Every error returns the same body:

```
{
  "error": "reader_error",
  "message": "Reader is not connected (state=CLOSED)",
  "readerCode": "MT_HARDWARE_ERR",
  "timestamp": "2026-08-27T09:14:02.118Z"
}
```

`error` is a stable machine-readable slug — branch on it. `message` is for humans and may change between versions; do not parse it. `readerCode` appears only when the failure came from the RFID module itself and is useful when reporting a problem to IntelliRail.

Status	Meaning	Typical cause	Retry?
200	OK	Request completed and the body carries the result	
202	Accepted	An asynchronous read was started; the result will be POSTed to your callback	
400	Bad request	Malformed body, invalid EAN, or a required field omitted	No — fix the request
401	Unauthenticated	Missing or invalid API key	No
403	Forbidden	Valid key, but it does not hold the scope this endpoint needs	No
409	Conflict	The reader is not in a state to do this: not connected, already reading, or armed in another mode	Yes, with backoff
500	Server error	Unexpected. Report it to the operator	Yes, with backoff

Why 409 and not 500. Almost every reader failure means the reader is not in a state where it can do that — not connected, already reading, no antenna. That is a conflict with current state rather than a server defect, so it returns 409. This keeps genuine faults visible in your 5xx alerting instead of drowning them.

3.2 Authentication

Every request must carry an API key. There are no anonymous endpoints.

3.2.1 Transport

For this deployment the API is served over plain HTTP at `http://<reader-host>:8081`. The reader and the WMS sit on the warehouse LAN, and that network is the security boundary.

3.2.2 The API key

Send your key in the `X-API-Key` header on every request. The quickest way to confirm connectivity and your key is the reader status endpoint, which needs nothing else:

```
curl http://<reader-host>:8081/api/v1/reader/status \
-H "X-API-Key: BpB5DJyg0I2C3sH8aw_-oFltc_i9aXQNZ1uLq5itatY"
```

Keys are 32 bytes of random data, URL-safe Base64 encoded. Treat one as you would a password: it is not an identifier, and it authenticates anyone who holds it.

The reader stores only the SHA-256 of each key, never the key itself, so a configuration file that leaks — or that ends up in a support bundle — discloses nothing usable. The practical consequence is that a lost key cannot be recovered, only revoked and reissued.

3.2.3 Scopes

Each key carries one or more scopes. They exist because a reader has consumers with genuinely different risk profiles, and a warehouse application has no business writing EPCs onto tags or changing radio power.

Scope	Permits
INVENTORY	Arm and disarm modes, run reads, and read reader status. This is what the two reading modes need.
COMMISSION	Write and lock tag memory. This is what Managed Writing Mode needs. Destructive, and a permanent lock cannot be undone.

Ask for two keys, not one. Use an INVENTORY-scoped key for the reading modes, which run continuously in production, and a separate COMMISSION-scoped key held only by whatever performs tag commissioning. A bug or a compromise in the production read path then cannot write to tags. Neither key should hold ADMIN.

3.2.3.1 Scope required by each endpoint

The first matching rule applies. Any endpoint not listed requires ADMIN, so an endpoint added in a future version is closed by default rather than accidentally open.

Method	Path	Scope	Used by
POST	/api/v1/mode	INVENTORY	Super Fast Mode — arm and disarm
GET	/api/v1/mode	INVENTORY	Super Fast Mode — read current arming
POST	/api/v1/inventory	INVENTORY	Managed Reading Mode
GET	/api/v1/reader/status	INVENTORY	Connectivity check and monitoring
POST	/api/v1/tags/write	COMMISSION	Managed Writing Mode

3.2.4 Authentication failures

Status	error	Meaning and what to do
401	missing_api_key	No X-API-Key header. Add it.
401	invalid_api_key	The key is unrecognised or has been revoked. Contact the operator.

Status	error	Meaning and what to do
403	insufficient_scope	The key is valid but lacks the scope for this endpoint. The message names the scope required.

None of the three is retryable. Retrying an authentication failure will never succeed and will fill the reader's log.

```
{
  "error": "insufficient_scope",
  "message": "This endpoint requires the COMMISSION scope. Your key holds: [INVENTORY].",
  "timestamp": "2026-08-27T09:14:02.118Z"
}
```

3.2.5 Authenticating the reader to the WMS

Super Fast Mode, and Managed Reading Mode with timed set to false, have the reader POST results into the WMS. That reverses the direction of trust: the WMS must be able to tell a genuine result from anything else on the LAN that can reach its endpoint. This matters more, not less, without TLS.

The WMS issues a callback token. The reader sends it on every callback in the X-Callback-Token header, and it is configured once per reader alongside the callback URL:

```
POST /wms/rfid/inventory HTTP/1.1
Host: wms.warehouse.local:9000
Content-Type: application/json
X-Callback-Token: 8Qb2rLp4TxVnE7sKmA1yZcJ0dHfW6uGi
```

- The token is opaque to the reader. Generate at least 32 characters of random data; do not derive it from anything guessable such as the site name.
- The WMS must reject a request whose token does not match with 401, and must not process the body.
- The reader treats 401 and 403 from the WMS as non-retryable and raises an operator alarm rather than looping.
- Rotate the callback token on the same schedule as the API key.

3.2.6 Getting and rotating a key

Keys are issued by whoever operates the reader, not self-service. Ask for a key, state which scopes you need, and expect to be given it exactly once — it cannot be retrieved afterwards.

For rotation, the reader accepts multiple valid keys at the same time. That allows a clean handover: the operator adds your new key, you deploy it, and only then is the old one revoked. There is no window in which the integration is broken. Plan for rotation at least annually, and immediately if a key may have been exposed.

- Store the key outside your source tree — an environment variable or a secret manager, never a committed configuration file.
- Do not log it. Log the key id, for example `acme-wms`, if you need to correlate; that is not secret.
- Use a different key per environment, so revoking a compromised test key does not stop production.

3.3 Super Fast Mode

In Super Fast Mode the WMS makes one call to arm the reader, and the reader then runs autonomously until it is disarmed. Every carton that arrives in the RFID reading zone is read, released, and reported to the WMS without further instruction.

This mode assumes a single SKU with a fixed number of articles per carton for the duration of the run, which is what allows the `ean` and `expectedCount` to be supplied once rather than per carton. It is the fastest of the three modes because no HTTP round trip sits in the critical path: the optical sensor fires before any request would have arrived.

3.3.1 Arming the reader

POST /api/v1/mode Scope: INVENTORY

Field	Type	Required	Meaning
mode	string	Yes	SUPERFAST to arm, IDLE to disarm.
ean	string	Yes	The EAN of the SKU being processed. Validated; an unknown or malformed EAN is rejected with 400.
expectedCount	integer	Yes	Articles per carton. Used both to judge completeness and to end each read as soon as the count is reached.
callbackUrl	string	Yes	Absolute URL on the WMS that the reader POSTs each result to.
direction	string	No	IN or OUT. Defaults to the tunnel's configured direction. Stamped on every result.

```
POST /api/v1/mode
X-API-Key: <your key>
Content-Type: application/json

{
  "mode": "SUPERFAST",
  "ean": "8905527164445",
  "expectedCount": 40,
  "callbackUrl": "http://wms.warehouse.local:9000/wms/rfid/inventory"
}

200 OK
{
  "mode": "SUPERFAST",
  "ean": "8905527164445",
  "gtin14": "08905527164445",
  "expectedCount": 40,
  "callbackUrl": "http://wms.warehouse.local:9000/wms/rfid/inventory",
  "direction": "IN",
  "armedAt": "2026-08-27T09:12:44.006Z"
}
```

3.3.2 What happens for each carton

Once armed, the reader repeats the following cycle without WMS involvement. The target for the whole cycle is three to five seconds.

- The optical sensor detects a carton arriving in the RFID reading zone, and the reader energises the antennas.
- The reader inventories the carton, ending as soon as it has seen `expectedCount` distinct EPCs of the armed EAN
- The reader releases the carton from the reading zone.
- The reader POSTs the result to `callbackUrl`.

The release happens before the callback, not after. A slow or unavailable WMS endpoint therefore does not stall the conveyor — results are retried and, if the WMS stays unreachable, spooled to the reader's disk. Your endpoint being down costs you catch-up work, not throughput.

3.3.3 The result callback

The reader POSTs one result per carton. The same object is returned by Managed Reading Mode, so an integration can share one parser.

POST /wms/rfid/inventory

Content-Type: application/json

X-Callback-Token: <the token your WMS issued>

```
{
  "id": "8f2b1c44-9a0e-4d7b-b2a1-7c5e9d3f0a11",
  "sequence": 1188,
  "mode": "SUPERFAST",
  "direction": "IN",
  "ean": "8905527164445",
  "gtin14": "08905527164445",
  "startedAt": "2026-08-27T09:14:02.118Z",
  "endedAt": "2026-08-27T09:14:03.948Z",
  "durationMs": 1830,
  "stopReason": "COUNT_REACHED",
  "expectedCount": 40,
  "complete": true,
  "matched": {
    "count": 40,
    "tags": [
      {
        "epc": "30361F8CDC10F0000000413",
        "tid": "E2801190A50300481234ABCD",
        "serial": 1043, "rssi": -47, "reads": 12, "firstSeenMs": 214 }
      ]
    },
    "unexpected": [],
    "undecodable": []
  }
}
```

3.3.3.1 Result fields

Field	Type	Meaning
id	string	Result identifier (UUID). Stable for the life of the result — deduplicate on this.
sequence	integer	Monotonic and durable; survives a reader restart. Your cursor if you ever need to reconcile.
mode	string	The mode that produced this result.
direction	string	IN or OUT, as armed.
ean / gtin14	string	The SKU this read was performed against.
startedAt / endedAt	timestamp	When the read began and ended, from the reader's clock.
durationMs	integer	How long the read took. Watch this against your line rate.
expectedCount	integer	Articles expected, as armed.
complete	boolean	True when the number of matched articles equals expectedCount exactly.
matched.count	integer	Distinct articles of the expected EAN that were found.
matched.tags	array	One entry per article: epc, tid, serial, rssi, reads and firstSeenMs.
unexpected	array	Articles that decoded correctly but carry a different EAN — a picking error, or a neighbouring carton bleeding into the field.
undecodable	array	Tags that replied but are not SGTIN-96, for example the carton's own SSCC label. Reported, never silently discarded.

3.3.4 What your endpoint must do

- Return 2xx quickly. The reader treats a response slower than two seconds as a failure and will retry.
- Be idempotent. Deduplicate on id. A retry after your 2xx was lost in transit delivers the same result twice, and that is by design — at-least-once delivery is far simpler to make correct than exactly-once.
- Distinguish 4xx from 5xx honestly. The reader retries 5xx and timeouts with backoff, and does not retry 4xx.
- Not depend on ordering. Use `sequence` if you need to detect a gap; do not rely on timestamps to order results across readers, because each reader stamps them from its own clock.

3.3.5 Disarming and changing SKU

Send { "mode": "IDLE" } to the same endpoint to stop. GET /api/v1/mode returns the current arming, which is worth checking at WMS start-up — a reader that was left armed by a previous shift will keep reading and reporting against the old SKU.

Changing SKU means re-arming. There is no partial update: send a fresh arming call with the new EAN and count. Cartons of a different SKU that pass while the reader is armed are not lost — their articles are reported in unexpected, and complete will be false — but they are not a substitute for re-arming.

3.3.6 Errors specific to this mode

Status	error	Cause
400	invalid_ean	The EAN failed validation, or is not one configured for this site.
400	callback_url_required	mode is SUPERFAST but no callbackUrl was supplied.
409	mode_conflict	A Managed Reading call is in progress, or the reader is already armed with different parameters. Disarm first.
409	reader_not_connected	The RFID module is not responding. Raise it with the operator; do not loop.

3.4 Managed Reading Mode

In Managed Reading Mode the WMS controls the movement of the carton through all three zones and calls the reader once per carton to perform the read. Nothing is armed in advance, so the SKU and the expected quantity can differ from carton to carton.

POST /api/v1/inventory Scope: INVENTORY

Field	Type	Required	Meaning
ean	string	No	The EAN expected in this carton. Optional but strongly recommended: it is what lets the reader report a wrong-SKU article as unexpected. Omitting it does not discard tags, it only changes what is reported.
expectedCount	integer	No	Articles expected in this carton. Supplying it sets complete and lets the read end as soon as the count is reached.
timed	boolean	Yes	Selects the synchronous or asynchronous form. See below.
durationMs	integer	If timed	How long to read for. Required when timed is true; ignored when false.
callbackUrl	string	If not timed	Absolute URL on the WMS to POST the result to. Required when timed is false.
tid	boolean	No	Return the TID of each article. Defaults to false. See the note on cost in Section 3.1.

3.4.1 The timed flag

The two forms differ in when the WMS gets the result, and in how long the read takes.

timed	Response	How the read ends
true	200 with the full result, after durationMs has elapsed	Always runs for the whole of durationMs, whether or not the expected count was reached earlier.
false	202 immediately with the result id; the result is POSTed to callbackUrl	Best effort within the reader's configured window, ending as soon as expectedCount distinct articles are found.

timed set to false is usually the faster of the two, and it is the one to prefer if your PLC can accept an asynchronous result. It ends the moment the expected count is reached, which on a well-read carton is often well under two seconds, whereas timed true always consumes the full window you asked for. Use timed true when the carton must be held for a deterministic period regardless of how quickly the tags are found.

3.4.2 Synchronous form

```
POST /api/v1/inventory
X-API-Key: <your key>
Content-Type: application/json

{
  "ean":          "8905527164445",
  "expectedCount": 40,
  "timed":        true,
  "durationMs":   3000,
  "tid":          true
}

200 OK (after 3000 ms)
{ ... the result object defined in Section 3.3 ... }
```

3.4.3 Asynchronous form

```
POST /api/v1/inventory
X-API-Key: <your key>
Content-Type: application/json

{
  "ean":          "8905527164445",
  "expectedCount": 40,
  "timed":        false,
  "callbackUrl":  "http://wms.warehouse.local:9000/wms/rfid/inventory",
  "tid":          true
}

202 Accepted
{
  "id":           "3c7a9e10-2b44-4f81-9d05-6ea2b8c41f77",
  "startedAt":    "2026-08-27T09:21:11.402Z"
}
```

The result arrives at callbackUrl as a POST carrying the x-Callback-Token header, in exactly the form shown in Section 3.3, with "mode": "MANAGED_READ" and the same id the 202 returned. Match on that id to tie the callback back to the carton you dispatched.

The requirements on your endpoint are the same as for Super Fast Mode: return 2xx quickly, deduplicate on id, distinguish 4xx from 5xx, and do not depend on ordering.

3.4.4 Worked example — a short carton

A carton declared as forty articles passes through an inbound tunnel. Two articles are missing. The reader is called with the manifest quantity:

```
{ "ean": "8905527164445", "expectedCount": 40, "timed": false,
  "callbackUrl": "http://wms.warehouse.local:9000/wms/rfid/inventory" }
```

The result that arrives at the callback:

```
{
  "id": "3c7a9e10-2b44-4f81-9d05-6ea2b8c41f77",
  "sequence": 1189,
  "mode": "MANAGED_READ",
  "ean": "8905527164445",
  "durationMs": 4284,
  "expectedCount": 40,
  "complete": false,
  "matched": { "count": 38, "tags": [ ... ] },
  "unexpected": [],
  "undecodable": [ { "epc": "31...", "reason": "header 0x31 is not SGTIN-96" } ]
}
```

This is a clean read of a short carton. The reader did its job correctly and the discrepancy is physical: two articles are not in that carton, or their tags have failed. Because the result arrived while the carton was still at the tunnel, it can be resolved now rather than at a stock count in six months.

Had stopReason been TIMEOUT with the same count of 38, the conclusion would be entirely different: the reader ran out of time with tags still arriving, and 38 would be a lower bound rather than a finding. Do not treat the two the same way, and do not write a short count into your stock record on the strength of a TIMEOUT.

3.4.5 Errors specific to this mode

Status	error	Cause
400	callback_url_required	timed is false but no callbackUrl was supplied.
400	duration_required	timed is true but durationMs was not supplied.
409	read_in_progress	A read is already running. A tunnel reads one carton at a time — that is a physical constraint, not a simplification. Serialise your callers.
409	mode_conflict	The reader is armed for Super Fast Mode. Disarm it before making managed calls.
409	reader_not_connected	The RFID module is not responding. Raise it with the operator; do not loop.

3.5 Managed Writing Mode

In Managed Writing Mode the WMS commissions tags: it passes the EAN and the number of tags to be written, and the reader encodes each tag it finds in the field and returns what it wrote. This is how blank or supplier-supplied tags are turned into articles your stock system recognises.

This mode uses the COMMISSION scope and it is destructive. Writing replaces whatever EPC a tag carried, and locking is not always reversible. It stops and restarts inventory around each operation, so it is for a commissioning station rather than a tunnel that is in service. Do not call it on a live line.

POST /api/v1/tags/write Scope: COMMISSION

Field	Type	Required	Meaning
ean	string	Yes	The EAN to encode. Validated; an unknown or malformed EAN is rejected with 400.
count	integer	Yes	How many tags to write. Also the quantity the response reports against.
startSerial	integer	No	First serial to use. 0, or omitted, means the reader allocates from its own counter. See below.
lock	boolean	No	Lock the EPC bank after writing. Defaults to false.
accessPassword	string	If lock	Four-byte access password as eight hex characters. Required when lock is true, rejected when it is false.

```
POST /api/v1/tags/write
X-API-Key: <your COMMISSION key>
Content-Type: application/json
```

```
{
  "ean": "8905527164445",
  "count": 40
}
```

200 OK

```
{
  "ean": "8905527164445",
  "gtin14": "08905527164445",
  "serialSource": "LOCAL",
  "requested": 40,
  "quantityMet": true,
  "written": [
    { "serial": 1073742891,
      "epc": "30361F8CDC100F004000042B",
      "tid": "E2801190A50300481234ABCD" }
  ],
  "failed": [],
  "localSerialNext": 1073742931
}
```

3.5.1 Response fields

Field	Type	Meaning
serialSource	string	LOCAL when the reader allocated the serials, CALLER when startSerial was supplied. Logged on every write so a collision can be traced.
requested	integer	The count you asked for, echoed back.
quantityMet	boolean	True when the number of tags successfully written equals requested. This is the flag to branch on.
written	array	One entry per tag written: the serial allocated, the resulting EPC, and the tag's TID. Keep the EPC-to-TID pairing — it is what later lets you detect a cloned tag.
failed	array	One entry per tag that could not be completed, carrying the stage reached and the reason. A non-empty failed array with quantityMet false is the normal partial outcome.
localSerialNext	integer	The next serial the reader's own counter will issue. Absent when serialSource is CALLER.

3.5.2 How the EPC is built

The EPC is encoded as SGTIN-96 from the EAN you supply and the serial that is allocated. The company prefix, its length, the GS1 partition value and the filter value are reader configuration set at commissioning, not parameters on this call.

The partition is deliberately not an API parameter. It cannot be derived from an EAN alone, and a wrong partition still decodes back to the correct GTIN — so a mistake would produce tags that look right to a decoder, are wrong on the wire, and give no visible symptom. It is fixed once, per site.

3.5.3 Serial allocation

startSerial	Behaviour
0' or omitted	Serials come from the reader's own counter. The counter is global to the reader, not per EAN, which is strictly stronger than GS1 requires. Serial 0 is never issued, so a blank or half-written tag reading as zeros can never be mistaken for a valid article.
greater than 0	Serials are startSerial through startSerial + count - 1. The reader's own counter is neither read nor advanced. The WMS owns that range entirely, including the risk of colliding with anything the reader allocated in the past.

One of the two must be the authority for a given site — decide which before commissioning begins, and do not mix them on the same reader. Where the reader allocates, its serial space is partitioned by reader identity, so several commissioning stations can run at different sites without coordinating.

3.5.4 Locking

- Setting `lock` true requires `accessPassword`, eight hex characters, and a password of all zeros is rejected. A Gen2 lock with the zero password is not a lock — anyone can unlock and rewrite the tag.
- Whoever holds that password holds the only ability to ever correct a tag. Decide where it lives before the first locked tag is written.
- The reader does not permanently lock tags. A permalock is irreversible: a mis-encoded tag becomes scrap, and if it is already on saleable stock, so is that article's tag.

The write sequence is: write the EPC, read it back and verify, set the access password, lock the EPC bank, then confirm the lock state. A failure at any stage returns that tag in failed with the stage it reached, so a partially commissioned tag is visible rather than silently counted as written.

Verification matters more than it looks. A write can report success and still have landed on a tag at the edge of the field that lost power part-way through the transaction. Catching that at the commissioning station is far cheaper than discovering it at the tunnel, or at a customer.

3.5.5 Errors specific to this mode

Status	error	Cause
400	<code>invalid_ean</code>	The EAN failed validation, or is not one configured for this site.
400	<code>access_password_required</code>	<code>lock</code> is true but no <code>accessPassword</code> was supplied, or it was all zeros.
400	<code>access_password_unexpected</code>	<code>accessPassword</code> was supplied but <code>lock</code> is false.
403	<code>insufficient_scope</code>	The key does not hold COMMISSION. This endpoint is not reachable with an INVENTORY key.
409	<code>reader_busy</code>	The reader is armed for Super Fast Mode or a read is in progress. Commissioning must not run against a line in service.

